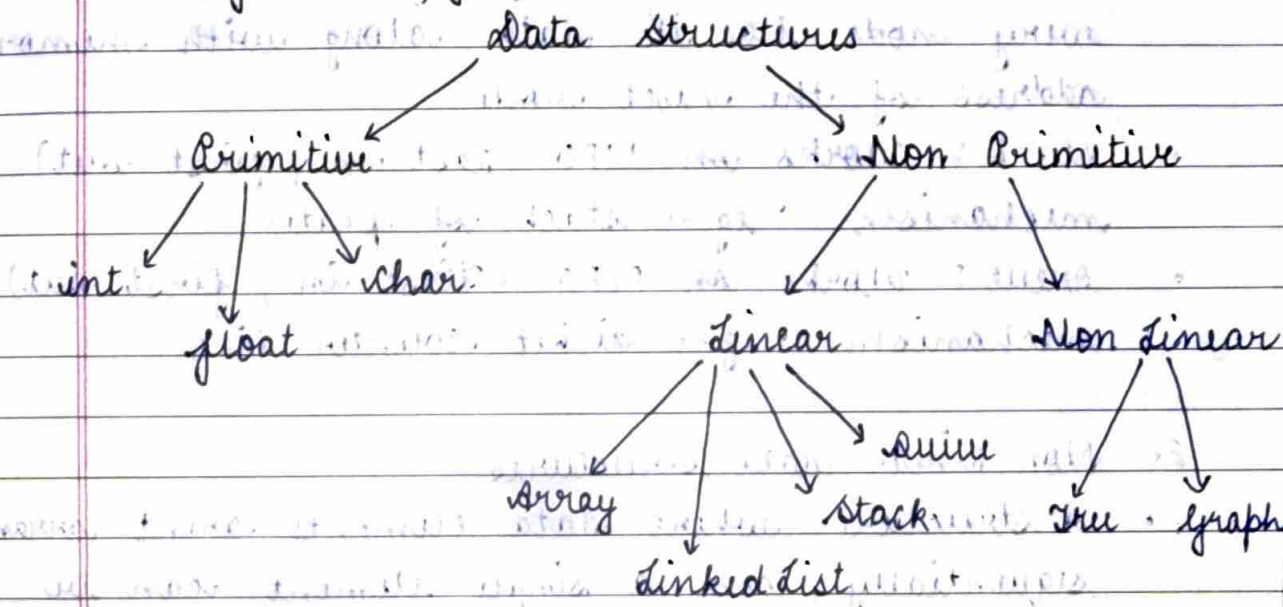


Unit-1→ Data structure

- A data structure is a specialized format, logical organisation and mathematical model used for organising, processing, retrieving and storing data in a computer's memory so that operations can be performed efficiently.

→ Classification of data structures→ Primitive data structures

- They are the fundamental, pre-defined data types directly supported by machine architecture and programming languages to hold single atomic values.

• eg → int, float, char, boolean, pointer

→ Non Primitive data structures

- They are the combinations of primitive data types and are complex, user-defined data structures derived from primitive data types.

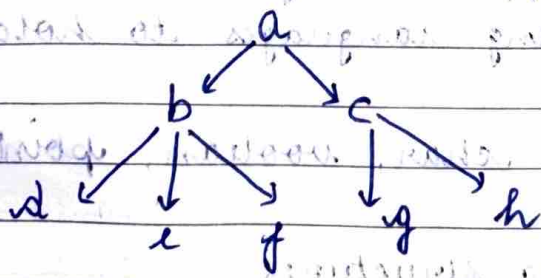
that can store collections of homogeneous or heterogeneous data items.

1. Linear Data Structures

- A linear data structure is the structure in which data items are arranged sequentially or linearly, where each element is attached to the previous & next (adjacent) element.
- Array: contiguous (continuous) memory block of fixed size and same data types.
- Linked list: collection of dynamic nodes where every node has its data along with memory address of the next node.
- Stack: works on LIFO (Last in, first out) mechanism. eg → stack of plates.
- Queue: works on FIFO (First in, first out) mechanism. eg → ticket counter line.

2. Non Linear data structures

- A structure where data elements aren't arranged sequentially, and a single element can be connected to multiple other elements representing hierarchical/network relationships.
- Tree: Hierarchical structure, like parent-child relations.



- Graph: Network of nodes (vertices) and connections (edges).

→ Data Structure's operations

- Actions that can be taken on the data structure are known as data structure operations. It includes :-
 1. **Traversing**: Accessing or visiting each element of a data structure exactly once, in order to process / display it.
 2. **Insertion**: The process of adding a new data element at a specific position within the data structure.
 3. **Deletion**: The process of removing an existing data element from a specified location in the data structure.
 4. **Searching**: Finding the location / memory address of a target element from a data structure.
 5. **Sorting**: Arranging the elements in a specific logical sequence either in ascending / descending order.
 6. **Merging**: Combining the elements of two separate data structures into a single consolidated data structure.

→ Algorithm complexity

- There are many ways (algorithms) to solve a problem, but to find the best method we check how much time it takes (time complexity) and the space it takes (space complexity) based on the number of steps / operations increased as the input size (n) is increased.
- Algorithm complexity is a measure of amounts of computational resources (time & space) that an algorithm requires as a function of input size (n).

1. Time complexity

The increase in amount of time taken increases as a function of input size (n)

for eg → linear search of array

```
for (i = 0; i < n; i++)
```

```
    if (arr[i] == target)
```

```
        return i;
```

$n = 1$, loop runs 1 times

$n = 10$

→ 10 times

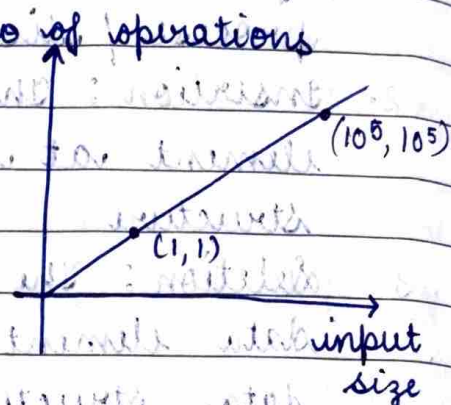
$n = 10^5$

→ 10^5 times

$$\Rightarrow f(n) = n$$

$$O(n)$$

$$f(x) = x$$



→ Big O Notation

- gives the no of atmost operations required in worst case scenario (upper bound)

→ How to find time complexity?

1. Ignore constants
2. Consider largest term

$$\text{eg } \Rightarrow \text{ If } f(n) = 5n^2 + \sqrt{n} + 100$$

$$\Rightarrow 1 + n^3 + \sqrt{n}$$

$$\Rightarrow \text{Time complexity} = O(n^2)$$

→ Big Omega Notation (Ω)

- Represents the lower bound of an algorithm's running time i.e. in the best case scenario or minimum number of operations

→ Big Theta Notation (Θ)

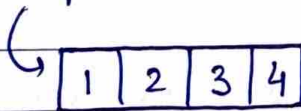
- It defines a tight bound on the execution time of an algorithm bounding it from both above and below
- It considers the average case scenario

2; Space complexity

- Amount of space taken by an algorithm as a function of input size (n)

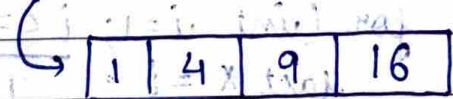
for eg :-

1. input $\rightarrow$ array $[\]$

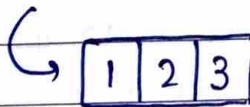


$O(n)$

output $\rightarrow$ array of squares

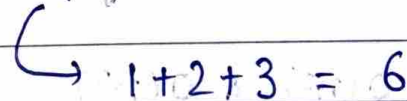


2. input $\rightarrow$ array $[\]$

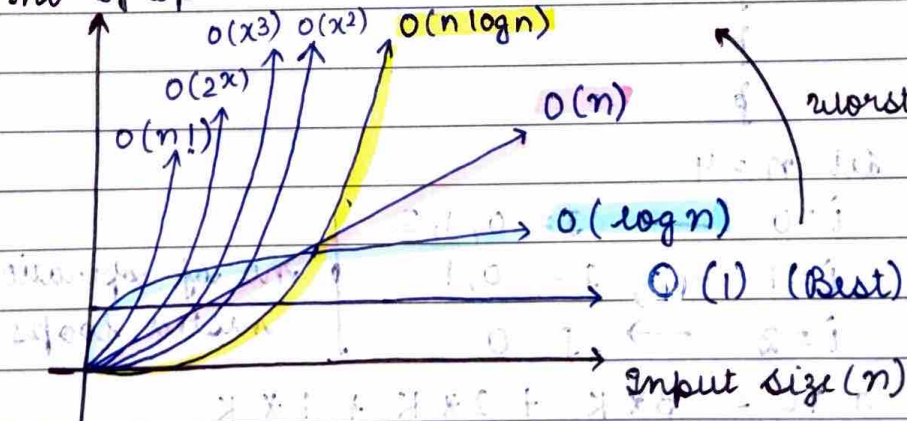


$O(k)$

output $\rightarrow$ sum of elements



no of operations



TIME COMPLEXITY

$O(1) > O(\log n) > O(n) > O(n \log n) > O(x^2) > O(x^3) > O(2^n) > O(n!)$

Best case

Worst case

→ Common time complexities

1. $O(1)$ → CONSTANT

• Sum of numbers from 1 to n
 $\text{int } n;$

$\text{int ans} = n * (n+1) / 2;$

→ Whatever is value of n , the formula is same & is executed in same no. of steps

2. $O(n)$ → LINEAR

• Factorial of a no.

$\text{int fact} = 1;$

for ($\text{int } i = 1; i \leq n; i++$) {

$\text{fact} * i;$

↳ depends on n

• Fibonacci series

• Kadane's algorithm

3. $O(n^2)$ & $O(n^3)$

• Bubble sort

for ($i = 0; i < n-1; i++$) {

for ($j = 0; j < n-i-1; j++$) {

if ($\text{arr}[j] > \text{arr}[j+1]$) {

swap ($\text{arr}[j], \text{arr}[j+1]$);

}

}

}

let $n = 4$

$i = 0$ (loop) $j = 0, 1, 2$

$i = 1$ (loop) $j = 0, 1$

$i = 2 \rightarrow j = 0$

no. of operations in nested loops

→ $TC = 3 * K + 2 * K + 1 * K$

For n , $TC = 3(n-1) * K + (n-2) * K + \dots + 1 * K$

$$\Rightarrow TC = \frac{n(n-1)K}{2} \quad (T.F) \quad \frac{n^2 K}{2} \approx \frac{n^2}{2}$$

$$\Rightarrow TC = O(n^2)$$

- $O(n^3) \rightarrow$ Three nested loops, for eg to print all the sub arrays of an array.

$$4. O(\log n)$$

Binary search $S \rightarrow$ start of array

$e \rightarrow$ end of array

int $s = 0$, $e = n-1$;

while ($s \leq e$) {

int $mid = s + (e-s)/2$;

if ($arr[mid] < target$) {

$s = mid + 1$;

else if ($arr[mid] > target$) {

$e = mid - 1$;

else {

return mid ;

}

}

$$\frac{n}{2^0} \rightarrow \frac{n}{2^1} \rightarrow \frac{n}{2^2} \rightarrow \frac{n}{2^x} = 1$$

$$\frac{n}{2^x} = 1$$

$$\Rightarrow \log n = x \log 2$$

$$\Rightarrow x = \log_2 n$$

$$T.C = O(\log n)$$

$$5. O(n \log n)$$

• sortings $\rightarrow$ merge sort, quick sort

• greedy algorithm

→ Abstract data type (ADT)

- It's a mathematical model / specifications for data types where the data type is defined by its behaviour from the point of view of a user, specifically in terms of possible values and possible operations, independently of any specific implementation.
- eg → while driving a car, we know that pressing brakes would stop the car, but we don't know the mechanism of hydraulics that happens in engine to stop the car as it's hidden from us (users). Here, the brake pedals are ADT (interface).

→ Advantages of ADT

1. Abstraction & information hiding
 - Complex low level unrequired implementation details are hidden from the user.
2. Reusability & Modularity
 - Once ADT contract is implemented, the source code could be changed multiple times without affecting the main application.
3. Flexibility to change implementation
 - If we implement stack using array today and found that it's easier to do the same using linked list tomorrow so we can directly change the main code.

→ Components of ADT

1. Declaration of data / values
 - It tells what to store

2. Declarations of operations (interface)

- Includes function definitions, input parameters & return types
() `pop()`, () `push()`, () `peek()`.

3. Axioms / Rules

- Includes the pre & post conditions required for the operation to be performed
- eg → we can't use `pop()` operation if stack is empty

→ Difference between data type, ADT & data structure

1. Data type

- At the programming language level
- Focuses on memory representation & built in operations
- It's pre-defined in compiler
- eg → `int`, `float`, `double`

2. ADT

- Based on theoretical / specification level
- Focuses on what operations can be performed
- Visibility is based on conceptual / class interface
- eg → `stack ADT`, `list ADT`, `queue ADT`

3. Data structure

- Based on implementation level
- Focuses on how data is organised in memory and algorithms used
- It's visible in actual source code
- eg → `array based stack`, `doubly linked list` and `binary search tree`

→ Standard examples of ADT

1. List ADT

- insert(), delete(), get(), size()

2. Stack ADT

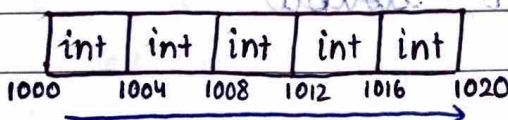
- push(), pop(), peak(), top(), isEmpty()

3. Queue ADT

- enqueue(), dequeue(), front(), isEmpty()

→ Array

An array is a linear data structure that contains a collection of homogeneous (similar data types) elements stored at continuous memory locations where each element can be randomly accessed using an integer index.



continuous memory allocation

→ Advantages of Array

- Random access to any element with a time complexity $O(1)$ which is very good.
- It provides memory efficiency for data because the memory is continuous so we don't need to take up space for pointers or addresses (as in linked list).
- It's cache friendly because CPU pre-fetching is very fast due to continuous memory allocation.

→ Applications of Array

- It helps in implementation of other structures as it acts as basic structure for stack, queue, heaps, hash tables & matrices.
- It can be used for look up tables as data retrieval is very fast.
- It can be used to perform sorting algorithms like quick sort, bubble sort, merge sort etc.

→ Implementation of Array

```
# include <stdio.h>
```

```
# define N 5
```

```
void display (int arr[], int size) {
```

```
    if (size == 0) {
```

```
        printf ("Array is empty");
```

```
    }
```

```
    else {
```

```
        printf ("Array elements :");
```

```
        for (int i = 0; i < size; i++) {
```

```
            printf ("%d\t", arr[i]);
```

```
        }
```

```
    }
```

```
int insert (int arr[], int *size, int element,  
            int pos) {
```

```
    if (*size >= N) {
```

```
        printf ("Array is full");
```

```
    }
```

```
    else if (pos < 0 || pos > *size) {
```

```
        printf ("error! invalid insertion");
```

```
    }
```



```
for (int i = *size; i > pos; i--) {
```

```
arr[i] = arr[i-1];
```

```
arr[pos] = element;
```

```
(*size)++;
```

```
}
```

```
int delete (int arr[], int *size, int pos) {
```

```
if (*size == 0) {
```

```
printf ("underflow");
```

```
}
```

```
else if (pos < 0 || pos >= *size) {
```

```
printf ("invalid position");
```

```
}
```

```
for (int i = pos; i < *size-1; i++) {
```

```
arr[i] = arr[i+1];
```

```
}
```

```
(*size) --;
```

```
}
```

```
int main () {
```

```
int arr[N];
```

```
int size = 0, choice, element, pos;
```

```
printf ("enter initial no of elements");
```

```
scanf ("%d", &size);
```

```
if (size < 0 || size > N) {
```

```
printf ("invalid size");
```

```
}
```

```
else {
```

```
printf ("enter %d elements \n", size);
```

```
for (int i = 0; i < size; i++) {
```

```
scanf ("%d", &arr[i]);
```

```
}
```

```
while (1) {  
    printf ("Operations Menu :-");  
    printf ("1. Display");  
    printf ("2. Insert");  
    printf ("3. Delete");  
    printf ("4. Exit");  
    printf ("Enter your choice from 1 to 4");  
    scanf ("%d", & choice);
```

```
    switch (choice) {
```

```
        case 1:
```

```
            display (arr, size);  
            break;
```

```
        case 2:
```

```
            printf ("Enter element to insert");  
            scanf ("%d", & element);  
            printf ("Enter position (0 to %d)", size);  
            scanf ("%d", & pos);
```

```
            if (insert (arr, *size, element, pos)) {  
                printf ("element inserted successfully");  
                p  
            }  
            break;
```

```
        case 3:
```

```
            printf ("Enter position to delete (0 to %d)", size-1);  
            scanf ("%d", & pos);  
            if (delete (arr, *size, pos)) {  
                printf ("element deleted successfully");  
                p  
            }  
            break;
```


case 4 :

```
printf ("Exiting program");
exit 0;
```

Default :

```
printf ("invalid choice, choose between 1 to 4");
```

```
{ char *msg = "Invalid choice\n";
```

```
{
```

```
return 0;
```

```
}
```

```
{ char *msg = "Invalid choice\n";
```

```
{ user
```

```
{ (msg, user) printf
```

```
{ user
```

```
{ user
```

```
{ ("Invalid choice\n") printf
```

```
{ ("Invalid choice\n") printf
```

```
{ (msg, "Invalid choice\n") printf
```

```
{ (msg, "Invalid choice\n") printf
```

```
{ (msg, "Invalid choice\n") printf
```

```
{ ("Invalid choice\n") printf
```

```
{
```

```
{ user
```

```
{ user
```

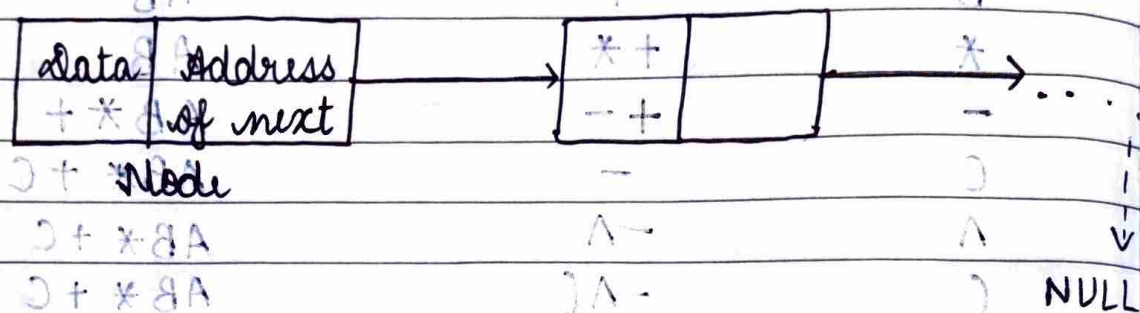
```
{ ("Invalid choice\n") printf
```

```
{ (msg, "Invalid choice\n") printf
```

```
{ (msg, "Invalid choice\n") printf
```

→ Linked List

- A linked list is a linear data structure where elements are stored in non contiguous memory locations and linked together using pointers.



- It uses dynamic memory allocation of heap memory.
- Linked lists are dynamic and grow & shrink at runtime.

→ Advantages of linked lists

- Dynamic memory allocation prevents memory waste.
- Fast insertion & deletion without element shifting.
- Effective execution for building stacks & queues.

→ Disadvantages of linked lists

- No random / indexed access.
- Extra memory overhead for storing pointers.
- Unidirectional only (can't traverse backward).

→ code for singly linked list

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node * next;
```

```
};
```

```
struct Node * head = NULL;
```

```
void insertAtHead (int val) {
```

```
    struct Node * newNode = (struct Node *) malloc (
```

```
        sizeof (struct Node));
```

```
    newNode → data = val;
```

```
    newNode → next = head;
```

```
    head = newNode;
```

```
    printf ("Inserted %d at beginning", val);
```

```
}
```

```
void insertAtTail (int val) {
```

```
    struct Node * newNode = (struct Node *) malloc (sizeof (
```

```
        struct Node));
```

```
    newNode → data = val;
```

```
    newNode → next = NULL;
```

```
    if (head == NULL) {
```

```
        head = newNode;
```

```
        return;
```

```
    }
```

```
    struct Node * temp = head;
```

```
    while (temp → next != NULL) {
```

```
        temp = temp → next;
```

```
    }
```

Phy List

```
temp → next = newNode;
printf ("%d inserted at end", val);
}
```

```
void insert At Position (int val, int pos) {
if (pos == 1) {
insert At Head (val);
return;
}
```

```
struct Node * newNode = (struct Node *) malloc (sizeof (
struct Node));
```

```
newNode → data = val;
```

```
struct Node * temp = head;
for (int i = 1; temp != NULL && i < pos; i++) {
temp = temp → next;
}
```

```
if (temp == NULL) {
printf ("invalid position");
}
```

```
free (newNode);
return;
```

```
newNode → next = temp → next;
```

```
temp → next = newNode;
printf ("%d inserted at position %d", val, pos);
}
```

```
void delete From Head () {
```

```
if (head == NULL) {
```

```
printf ("list is empty");
return;
```

```
}
```



```

1) struct Node * temp = head; // 1. train
   head = head -> next; // delete = delete
   printf ("%d deleted from beginning", temp->data);
   free (temp); // delete || NULL == delete ||
   }

```

```

void deleteFromTail () {

```

```

    if (head == NULL) {

```

```

        printf ("list is empty");

```

```

        return;

```

```

    if (head -> next == NULL) {

```

```

        printf ("%d deleted from end", head->data);

```

```

        free (head);

```

```

        head = NULL;

```

```

        return;
    }

```

```

    struct Node * temp = head;
    while (temp -> next -> next != NULL) {
        temp = temp -> next;
    }

```

```

    printf ("%d deleted from end", temp->next->data);

```

```

    free (temp -> next);

```

```

    temp -> next = NULL;

```

```

}

```

```

void deleteFromPos (int pos) {

```

```

    if (head == NULL) {

```

```

        printf ("list is empty");

```

```

        return;
    }

```

```

}

```

```

struct Node * temp = head;

```



```
for (int i = 1; temp != NULL & i < pos + 1; i++) {
```

```
temp = temp -> next;
```

```
if (temp == NULL || temp -> next == NULL) {
```

```
printf("invalid position");
```

```
return;
```

```
}
```

```
struct Node * nodeToDelete = temp -> next;
```

```
temp -> next = nodeToDelete -> next;
```

```
printf("%d deleted from %d position", nodeToDelete -> data,
```

```
pos);
```

```
free(nodeToDelete);
```

```
}
```

```
void display() {
```

```
if (head == NULL) {
```

```
printf("List is empty");
```

```
return;
```

```
}
```

```
struct Node * temp = head;
```

```
printf("List:");
```

```
while (temp != NULL) {
```

```
printf("%d -> ", temp -> data);
```

```
temp = temp -> next;
```

```
}
```

```
printf("NULL");
```

```
}
```

```
int main() {
```

```
insertAtHead(10);
```

```
insertAtEnd(9);
```

```
deleteFromHead();
```

```
display();
```

```
}
```